PRODUCT REQUIREMENTS DOCUMENT

High-Concurrency Serverless Event Platform

Transactional ticket safeguards, queue-buffered auto-scaling, and fraud-resilient digital ticketing

Document control	Value
Product stage	Greenfield platform blueprint
Primary audience	Product, engineering, security, operations, and compliance leaders
Version	1.0 | 19 August 2026
Decision status	Baseline requirements for discovery, architecture, and phased delivery

Executive Summary

This PRD defines a serverless-first event ticketing platform built for on-sale moments where demand can exceed inventory by orders of magnitude. The platform must preserve a zero-oversell inventory invariant while providing a fair, transparent, and responsive purchase experience to 100,000+ concurrent users.

The design combines a virtual waiting room, strict transactional reservation controls, idempotent payments, and event-driven recovery. Strongly consistent inventory commands are deliberately separated from highly scalable read models; every business-critical state transition is auditable, reversible where appropriate, and observable end to end.

Product Vision

Enable event organizers to sell scarce inventory fairly and reliably at global scale, while giving genuine fans confidence that availability, queue access, pricing, and order outcomes are accurate.

Goals

- Guarantee that a seat or ticket unit can be sold at most once, even under network retries, failures, and regional traffic surges.
- Absorb demand spikes without a platform-wide outage and keep customers informed of progress and recovery options.
- Deter automation and fraud without creating unreasonable friction for legitimate buyers.
- Give organizers live operational control, trustworthy inventory visibility, and a complete audit trail.
- Support phased rollout from general admission inventory to assigned seats, transfers, refunds, waitlists, and multi-region operation.

Non-Goals

- Operating as a merchant of record or storing raw payment-card data.
- Providing a general-purpose CRM, marketing automation suite, or venue-management system.
- Guaranteeing a queue position across a user changing identity, device, or browser without completing re-authentication.
- Allowing eventual consistency for authoritative inventory writes or purchase confirmation.

1. Problem Statement

Ticket sales for high-demand events are adversarial distributed systems problems: many clients concurrently attempt to purchase the same scarce resource while payments, identity checks, caching, queues, and notifications each have different latency and failure profiles. Conventional “check then decrement” flows permit stale reads, lost updates, double charging, and overselling; simple rate limits create unfair queues and bot workarounds.

1.1 Design Principles

- Inventory truth is singular and strongly consistent. Availability projections may be eventually consistent; reservation and purchase commands are not.
- A reservation is a lease, not a sale. It expires automatically and its release is idempotent.
- Every external side effect has an idempotency key, a durable command record, and a reconciliation path.
- Fairness is an explicit product policy, measured and monitored—not an accidental property of load balancing.
- Fail closed for inventory and payment integrity; fail soft for browsing, analytics, and nonessential personalization.
- Security and privacy controls are product requirements, not post-launch add-ons.

1.2 Core Risks Addressed

Failure mode	Required control	Expected outcome
Race conditions / oversell	Serializable inventory command, seat-level lease or atomic conditional decrement, version checks	One successful claim per ticket unit
Dirty / phantom reads	Authoritative write store with transactional isolation; CQRS projections labeled as advisory	No sale based on stale availability
Spike crash / timeout	Admission queue, queue-depth scaling, back-pressure, graceful read-only modes	Controlled load and recoverable UX
Duplicate payments	Payment idempotency key, durable payment intent state, provider reconciliation	At most one capture per order
Bots / scalpers	Risk scoring, velocity limits, progressive challenges, identity entitlement	Reduced automated inventory capture
Distributed inconsistency	Outbox/inbox patterns, saga state machine, replayable events, reconciliation	Detectable and compensable drift
Deadlocks	Single resource ordering, short leases, no long-held DB locks, retry policy	Bounded contention and recovery

2. Users and Personas

Persona	Primary needs	Success signal
Fan / buyer	Fair access, accurate availability, safe and simple checkout, reliable digital ticket	Completes purchase or receives a clear alternative without unexplained failure
Organizer / event admin	Configure events and inventory, control sales policies, see operations in real time	Can launch and manage an on-sale without engineering intervention
Venue operations manager	Map sections/seats, enforce capacity and accessibility rules, handle entry exceptions	Capacity and issuance records reconcile with venue plan
Trust & safety analyst	Detect bots, fraud, account takeover, transfer abuse, suspicious patterns	Investigates alerts with evidence and can apply proportionate controls
Support agent	Find order state, explain queue/payment outcomes, issue authorized refunds or transfers	Resolves customer issues without unsafe manual inventory changes
Platform SRE / engineer	Observe latency, contention, queues, DLQs, and business invariants	Can diagnose and mitigate an incident before customer harm

3. Scope

3.1 In Scope

- Public event discovery, search, browsing, event detail pages, and real-time availability indicators.
- Organizer dashboard for event configuration, ticket tiers, seat maps, sales windows, limits, pricing policy, and waitlists.
- Virtual waiting room with signed admission tokens, position/progress updates, and fairness policy controls.
- Reservation, checkout, payment, order confirmation, digital QR ticket issuance, transfers, refunds, and expiry.
- Fraud prevention controls and support tooling; observability, audit, replay, and reconciliation workflows.
- Serverless, event-driven platform architecture with CQRS, event sourcing for business audit, and multi-region readiness.

3.2 Out of Scope for Initial Release

- Physical access-control hardware integration beyond QR payload export/API contracts.
- Secondary-market exchange, auctions, or dynamic resale marketplace.
- Custom fraud-model training from proprietary data; initial release uses rules, vendor signals, and configurable thresholds.
- Complex bundled merchandise fulfillment, travel packages, and promoter revenue settlement.

3.3 Phased Delivery

Phase	Capability outcome	Exit criteria
0: Foundations	Identity, event catalog, inventory command service, audit events, observability baseline	Proof: concurrent reservation load test has zero duplicate allocations
1: Controlled on-sale	Queue, GA tiers, checkout/payment, receipts, basic fraud controls	Peak drill supports target concurrency and recovery runbook
2: Seat and lifecycle	Interactive maps, assigned seats, transfers, refunds, waitlists, dynamic pricing	Full customer lifecycle reconciles across projections and ledger
3: Global scale	Multi-region routing, predictive pre-warm, advanced risk signals, operator analytics	Regional failover drill preserves inventory correctness

4. Functional Requirements

4.1 Event Management and Inventory

ID	Requirement	Acceptance criteria
FR-EM-01	Admins can create events with timezone-aware schedule, sales windows, venue, policy, tax, currency, and status configuration.	Invalid or overlapping sales windows are rejected; all changes are audit logged.
FR-EM-02	Admins can define ticket tiers (VIP, General, Early Bird, accessible allocations) with prices, limits, and eligibility.	Tier limits cannot exceed authoritative capacity; policy changes take effect at documented boundary.
FR-EM-03	The platform supports general-admission pools and assigned section/seat inventory.	Each inventory unit has a stable ID, state, version, and event/venue ownership.
FR-EM-04	Admins can publish interactive seat maps and availability views.	Public UI shows advisory availability timestamp; command path remains authoritative.
FR-EM-05	Dynamic pricing rules use approved demand signals and explicit guardrails.	Every price quote contains rule/version, currency, expiry, and audit metadata.
FR-EM-06	Waitlists collect consent and priority attributes and offer released inventory automatically.	Offer has a lease and expiration; unclaimed inventory returns safely to sale.

4.2 Transactional Ticket Safeguards

ID	Requirement	Acceptance criteria
FR-TX-01	Reserve command must atomically validate eligibility, capacity, state, version, and purchase limit before creating a lease.	Concurrent attempts for one unit result in one lease or one purchase; all others receive conflict/unavailable.
FR-TX-02	Reservation TTL is configurable per event and auto-releases through durable expiry processing.	Expired lease cannot be paid or confirmed; release is idempotent and emitted once logically.
FR-TX-03	Use optimistic concurrency with inventory version tokens; retry only after fresh validation.	Stale version returns deterministic conflict and never silently overwrites.

ID	Requirement	Acceptance criteria
FR-TX-04	Where a distributed lock is used, it is fencing-token protected, short-lived, and never the sole correctness mechanism.	Commands reject stale fencing tokens; lock loss triggers safe abort.
FR-TX-05	Checkout converts a valid reservation to an order through a durable saga.	State transitions are append-only and compensations occur for failed capture/issue.
FR-TX-06	Every payment initiation, capture, refund, and webhook has an idempotency/deduplication key.	Retries produce same result and provider reconciliation flags mismatch.
FR-TX-07	Deadlock prevention uses canonical resource ordering and avoids holding inventory locks during remote calls.	Contention test completes without unbounded waits; timeouts return recoverable state.

4.3 Canonical Reservation-to-Purchase State Machine

1. Requested: authenticated buyer submits inventory selection, admission token, risk context, and idempotency key.
2. Reserved: transaction/conditional write creates a bounded lease and emits ReservationCreated via transactional outbox.
3. Payment authorized: payment intent is created or retrieved using the same key; inventory lease remains valid only to its expiry.
4. Purchased: capture success and order commitment are recorded atomically where supported, otherwise by saga with a durable command ledger.
5. Issued: QR ticket issuance event is consumed idempotently; confirmation is sent asynchronously.
6. Compensated / Expired: failed payment, risk rejection, or TTL expiry releases inventory exactly once and emits an audit event.

4.4 Queue-Buffered Auto-Scaling

ID	Requirement	Acceptance criteria
FR-Q-01	High-demand events can enable a virtual waiting room before sale opens.	Only users with valid signed admission tokens reach protected purchase endpoints.
FR-Q-02	Queue assignment is position-aware and fairness-policy based (arrival, lottery, entitlement, or hybrid).	Position, policy basis, and estimate are shown; priority changes are auditable.
FR-Q-03	Client receives live progress updates using SSE/WebSocket with polling fallback.	Updates degrade gracefully; stale connection does not lose queue place within policy grace period.
FR-Q-04	Admission rate adapts to downstream capacity, reservation contention, payment health, and error budgets.	Controller reduces admission before downstream saturation and recovers gradually.
FR-Q-05	Autoscaling uses queue depth, oldest-message age, concurrency, latency, and error metrics.	Scaling policy is tested against burst profiles and protected by concurrency caps.
FR-Q-06	Predictive pre-warming is scheduled from event calendars, historical demand, and manual overrides.	Warm capacity is available before sale window; cost guardrails and rollback are logged.
FR-Q-07	Starvation detection identifies cohorts that wait disproportionately long.	Alert and policy action occur when fairness SLO is breached.

4.5 Anti-Fraud and Bot Protection

- Apply layered rate limits by IP/network, account, device, identity, event, payment instrument, and action; use retry-after messaging.
- Escalate risk friction progressively: silent signals, proof-of-work/behavioral checks, CAPTCHA, step-up authentication, and manual review only as needed.
- Collect privacy-reviewed device and behavioral signals with documented retention; do not use opaque signals as the sole irreversible denial basis.
- Enforce configurable per-identity and household/ticket limits using verified identity and entitlement rules.
- Protect sessions with short-lived signed tokens, secure cookies, token binding where available, session rotation after login, and anomaly-triggered re-authentication.
- Issue digitally signed, rotating QR credentials and validate transfer/refund lifecycle at scan time through an online or offline policy.

4.6 Buyer Experience

- Responsive discovery and event detail pages with accessible filters, venue and ticket information, and availability status.
- Seat/section selection reflects read-model availability with an explicit “held while you checkout” state after successful reservation.
- Queue screen provides current position/estimated wait, accessibility-friendly updates, reconnect guidance, and clear no-guarantee language.
- Checkout minimizes steps, preserves selections after recoverable failures, and surfaces specific next actions—not generic errors.

- PWA supports mobile wallet-ready ticket delivery, offline viewing of an encrypted/signed ticket representation where policy allows, and push/email confirmation.
- Buyer dashboard supports ticket viewing, eligible transfer, refund request/status, security settings, and order history.

4.7 Monitoring, Support, and Operations

ID	Requirement	Acceptance criteria
FR-OP-01	Provide live dashboard for sold, reserved, available, expired, and disputed inventory counts by event/tier/section.	Counts show source, freshness, and reconciliation status.
FR-OP-02	Emit distributed traces across gateway, queue, reservation, payment, order, ticket, and notification paths.	A support/admin user can trace an order using correlation ID with access controls.
FR-OP-03	Detect anomalies: inventory drift, repeated payment errors, bot spikes, price rule anomalies, queue fairness breach, and DLQ growth.	Alerts include impact, runbook link, event context, and deduplicated severity.
FR-OP-04	Provide DLQ inspection, safe replay, poison-message quarantine, and replay audit.	Replay requires authorization, preserves idempotency, and reports resulting outcome.
FR-OP-05	Support agents can look up order state and perform policy-authorized refund/transfer actions.	All actions are RBAC protected, reason-coded, and append audit events.

5. Architecture and Data Requirements

5.1 Logical Architecture

Edge and identity services validate sessions, admission tokens, rate limits, and risk signals before a client reaches commands. The queue admission controller meters traffic into stateless serverless command handlers. An authoritative inventory service owns reservation and sale transitions in a strongly consistent transactional store. Payment, order, issuance, notifications, analytics, and read projections consume immutable domain events through a broker using inbox/outbox deduplication.

Layer	Responsibilities	Consistency expectation
Edge / queue	CDN, WAF, bot controls, admission tokens, progress updates	Best effort for display; durable queue placement
Command domain	Inventory, reservations, orders, entitlement validation	Strong consistency / serializable per aggregate
Event backbone	Outbox publication, broker routing, idempotent consumers, DLQ	At-least-once delivery with exactly-once business effect
Query domain	Availability, discovery, dashboards, analytics	Eventually consistent, freshness disclosed
External adapters	Payment, identity, notifications, maps	Circuit breaker, timeout, idempotency, reconciliation

5.2 CQRS and Event Sourcing

- Commands validate against the authoritative aggregate and append domain events such as InventoryConfigured, ReservationCreated, ReservationExpired, PaymentAuthorized, OrderPurchased, TicketIssued, TransferCompleted, and RefundCompleted.

- Read projections are rebuildable from events plus snapshots; projection lag is measured and exposed to operators.
- Use an outbox in the same transaction as authoritative state change; consumers use an inbox/deduplication store keyed by event ID and handler version.
- Event schema includes event ID, aggregate ID/version, correlation ID, causation ID, actor, timestamp, schema version, idempotency key reference, and privacy classification.
- Retain immutable audit events according to legal and privacy policy; support GDPR deletion through tokenization, minimization, encryption-key deletion, and redacted projections—not mutation of financial records.

5.3 Authoritative Inventory Invariants

- A unit is in exactly one state: Available, Reserved, Sold, Blocked, or Retired.
- For each unit, at most one active reservation lease exists, and Sold is terminal except for controlled refund/reissue workflows.
- A reservation references an inventory version/fencing token and has a deterministic expiry.
- Available + Reserved + Sold + Blocked + Retired reconciles to configured capacity for each tier/section, subject to documented pool adjustments.
- A payment capture may produce at most one purchasable order; any mismatch is quarantined for reconciliation.

5.4 Resilience and Graceful Degradation

Dependency condition	Platform behavior	Customer experience
Read projection delayed	Serve last-known availability with timestamp; commands validate truth	Browse continues; final availability confirmed at reserve
Queue realtime channel degraded	Fall back to signed polling cadence	Position retained and updates continue at lower fidelity
Payment provider degraded	Open circuit, stop new payment attempts, preserve eligible leases only within policy	Clear retry/rejoin path; no ambiguous charge message
Inventory command store impaired	Fail closed for reserve/purchase; retain queue and browsing	No oversell; explain temporary pause and preserve fairness policy
Notification provider impaired	Commit order/ticket independently; retry notification from event log	Confirmation available in dashboard even if email is delayed
Analytics/monitoring impaired	Continue critical commands with local safety alerts and buffered telemetry	No customer-facing impact unless safety threshold demands throttling

5.5 Multi-Region and Sharding Strategy

Inventory correctness takes precedence over write locality. Each event (or inventory shard) has a designated authoritative write region/partition at a time. Global edge routing and regional read projections minimize latency, while writes route to the authoritative shard with bounded timeout and explicit failover controls. Active-active writes for the same scarce inventory require a proven strongly consistent global database or deterministic single-writer ownership; otherwise they are prohibited.

- Shard by event ID, with hot-event isolation and separate queue/admission partitions to prevent noisy neighbors.
- Use seat/tier aggregate partitioning only when transactional boundaries preserve the capacity invariant; avoid cross-shard decrement patterns without a coordinator.
- Failover requires fencing/epoch change, drain or rejection of prior-writer traffic, replication verification, and an audited operator decision.
- Replicate encrypted projections and event logs according to residency requirements; restrict personal-data replicas by region and retention policy.

6. User Stories and Acceptance Criteria

ID	User story	Acceptance criteria
US-01	As a fan, I want to enter a fair waiting room so I know when I can purchase.	I receive a signed queue token, position/progress, reconnect grace, and accessible updates; bypass attempts are rejected.
US-02	As a fan, I want a selected ticket held briefly while I pay.	A successful reserve returns lease expiry; another buyer cannot buy it during the active lease.
US-03	As a fan, I want payment retries not to charge me twice.	Submitting same checkout idempotency key returns the same payment/order outcome with one provider capture.
US-04	As an organizer, I want to configure ticket tiers and sales rules.	Published configuration validates capacity, limits, eligibility, and timezones before activation.
US-05	As an organizer, I want live inventory visibility during a sale.	Dashboard distinguishes authoritative counts from projection freshness and highlights drift.
US-06	As a trust analyst, I want suspicious automation escalated fairly.	Rules create explainable risk decisions, rate/challenge actions, and reviewable evidence.
US-07	As a support agent, I want to explain an ambiguous checkout.	Order timeline shows correlation ID, payment state, reservation expiry, and safe next action.
US-08	As an SRE, I want an overloaded system to protect inventory.	Queue controller throttles admissions and circuit breakers stop cascading failure before command SLO breach.
US-09	As a ticket holder, I want a fraud-resistant QR ticket.	QR token is signed, lifecycle-aware, and invalidated/rotated after transfer, refund, or security event.
US-10	As a privacy officer, I want personal data handled lawfully.	Consent, purpose, retention, deletion requests, and regional data controls are auditable.

7. Non-Functional Requirements

Category	Requirement	Target / verification
Scale	Absorb peak on-sale audience without global failure.	100,000+ concurrent active users; documented load profile including bots and retries.
Performance	Serve availability and queue status quickly.	p95 availability read < 500 ms; p95 protected command acknowledgement < 1 s under normal admitted load.
Correctness	Never oversell authoritative inventory.	Zero duplicate allocations in production and chaos/load-test invariant checks.
Availability	Critical sales workflow is highly available.	99.99% monthly SLA for defined purchase-path components; published exclusions and maintenance policy.
Durability	Do not lose accepted commands or audit history.	Durable command/event ledger; tested backup, restore, and replay RPO/RTO targets.
Security	Protect identity, payment, and ticket credentials.	TLS 1.2+ in transit, encryption at rest, least privilege, secrets rotation, security testing.

Category	Requirement	Target / verification
Compliance	Meet payment and privacy obligations.	PCI DSS scope minimized through tokenized PSP integration; GDPR DPIA, DSR workflow, data minimization.
Accessibility	Support inclusive user journeys.	WCAG 2.2 AA target for buyer/admin UI; queue and challenge alternatives.
Observability	Detect integrity and capacity risk quickly.	SLO dashboards, trace coverage for critical path, actionable alerts and runbooks.
Recovery	Recover from partial and regional failure safely.	Regular failover, restore, payment reconciliation, and DLQ replay exercises.

8. Success Metrics

Dimension	Metric	Target / guardrail
Inventory integrity	Oversell incidents; duplicate unit allocation	0; automated invariant check on every authoritative transition
Checkout integrity	Duplicate charge rate; unreconciled payment/order mismatch	0 duplicate captures attributable to platform; mismatch aged < 15 min
Fairness	Queue cohort wait-time disparity; starvation events	Policy-defined disparity threshold; 0 unresolved starvation breaches
Reliability	Purchase-path availability; error rate; command timeout rate	99.99% SLA; error budget alerts before breach
Performance	p95 queue update latency; p95 availability read; p95 reserve acknowledgement	< 2 s; < 500 ms; < 1 s under admitted load
Conversion	Admitted-to-reserved, reserved-to-purchased, payment recovery	Baseline per event tier; improve without weakening fraud control
Fraud	Bot challenge precision, fraud loss, false-positive appeal rate	Thresholds set per event; human review sampling validates outcomes
Operations	Mean time to detect/mitigate integrity incident; DLQ oldest age	Event-specific SLOs; recurring game-day exercises
Customer trust	CSAT after queue/checkout; support contacts per 1,000 attempts	Improving trend; segmented by queue and failure outcome

9. Security, Privacy, and Compliance

9.1 Security Controls

- Use OIDC/OAuth 2.1 aligned authentication, MFA for privileged roles, RBAC/ABAC with least privilege, and just-in-time access for production operations.
- Encrypt PII, payment tokens, ticket secrets, and logs with managed keys; rotate keys and separate tenant/event access paths.
- Use WAF, DDoS mitigation, signed webhooks, request signing where appropriate, secrets vaulting, dependency scanning, and secure SDLC controls.
- Apply abuse protections with accessibility alternatives: challenge escalation must provide equivalent nonvisual and assisted paths.
- Log privileged and security-sensitive operations immutably; restrict audit-log access and monitor exfiltration.

9.2 PCI DSS Scope Strategy

Use a PCI DSS-compliant payment service provider hosted fields or redirect flow so the platform receives tokens, payment method references, and provider webhooks rather than PAN/CVV. Keep cardholder data out of logs, events, analytics, and support views. Formal scope, SAQ/ROC obligations, and shared-responsibility controls must be validated with the chosen PSP and a qualified assessor.

9.3 GDPR and Data Governance

- Document lawful basis and purpose per data category; obtain and record consent where required.
- Minimize collection of device/risk data; publish notices that explain categories, purpose, retention, and meaningful logic involved in automated decisions.
- Implement data subject access, correction, deletion, objection, and portability workflows with identity verification and legal-hold handling.
- Maintain records of processing, DPIAs for high-risk profiling, processor agreements, cross-border transfer safeguards, and breach-response procedures.

10. Constraints, Assumptions, and Dependencies

10.1 Constraints

- The zero-oversell guarantee requires a single authoritative transaction boundary or equivalent globally strongly consistent conditional-write primitive for each inventory aggregate.
- Serverless execution limits, cold starts, and regional quotas must be designed around through provisioned concurrency/pre-warming, asynchronous buffering, and strict concurrency budgets.
- Third-party payments, identity proofing, CAPTCHA, messaging, and mapping providers can fail or impose rate limits; adapters must be replaceable and circuit-broken.
- Queue fairness and anti-bot policies have legal, accessibility, and brand implications; product/legal approval is required before each major event policy.
- Dynamic pricing may be regulated and must be transparent according to jurisdiction and organizer commitments.

10.2 Assumptions

- Organizers provide authoritative venue capacity, tax configuration, ticketing terms, and any eligibility/identity rules before sale activation.
- A payment provider supports idempotency, authorization/capture lifecycle, signed webhooks, dispute events, and regional coverage required for the launch market.
- Buyers have a verified account or an event-appropriate identity flow for restricted high-demand windows.
- The chosen cloud supports managed queues, serverless concurrency controls, a strongly consistent transactional data store, KMS, observability, and multi-region deployment.
- Legal and compliance stakeholders define retention, data residency, refund, transfer, and accessibility requirements per launch region.

10.3 Open Decisions

Decision	Why it matters	Owner
Authoritative inventory store and regional write model	Determines transaction semantics, latency, sharding, and failover approach	Architecture / SRE
Queue fairness policy per event class	Balances equality, entitlement, accessibility, and anti-bot goals	Product / Legal / Trust
Identity proofing and ticket-limit policy	Determines fraud reduction, conversion, privacy, and support burden	Trust / Product
Payment provider and merchant-of-record model	Sets compliance scope, reconciliation and refund workflow	Finance / Compliance
Seat-map provider vs. native capability	Affects data ownership, UI complexity, and venue integration	Product / Engineering

11. Delivery and Validation Plan

11.1 Engineering Quality Gates

- Property-based and concurrency tests prove inventory invariants across retries, duplicate events, lease expiry races, and partial saga failures.
- Load tests simulate 100,000+ concurrent users, bot traffic, reconnect storms, cache misses, and payment latency.
- Chaos tests inject broker duplicates, delayed webhooks, regional latency, dependency outages, cold starts, and process termination during each state transition.
- Security tests include threat modeling, penetration testing, session and QR abuse testing, webhook forgery, RBAC tests, and supply-chain controls.
- Operational game days rehearse traffic ramps, queue policy changes, payments outage, inventory pause, DLQ replay, reconciliation, and region failover.

11.2 Launch Readiness Checklist

- Event capacity and state invariants reconciled; runbook owners and escalation paths staffed.
- Queue policy, admission limits, rate controls, and pre-warming schedule reviewed and approved.
- Dashboards, alerts, traces, audit search, and support tooling tested with realistic correlation IDs.
- Payment sandbox/production webhooks verified; idempotency and reconciliation reports pass.
- Accessibility, privacy notice, terms, support macros, status page, and graceful-degradation messaging reviewed.
- Rollback/pause controls tested; post-event reconciliation and incident review templates prepared.

12. Acceptance Definition

The platform is ready for a controlled production on-sale when it meets the non-functional targets under a representative peak-load test, demonstrates zero duplicate allocation and payment-capture behavior across failure injection scenarios, passes security/compliance readiness review, and completes an end-to-end operational game day with organizer, support, trust, and SRE participants. Any detected inventory drift, ambiguous payment state, or fairness SLO breach blocks expansion until reconciled and remediated.